

Human Activity Recognition from Wrist Accelerometer Data

Data Mining — Assignment 3

Student ID: 314540061

GitHub (public): <https://github.com/Lappykentang/DM2026-Assignment-3>

Kaggle public macro-F1: 0.8267

0. Code, and how to run it

All of my code is in the public GitHub repository below, and the link is repeated here as required:

<https://github.com/Lappykentang/DM2026-Assignment-3>

There are two equivalent ways to run everything. The first is the script pipeline, which I used day to day:

```
git clone https://github.com/Lappykentang/DM2026-Assignment-3
cd DM2026-Assignment-3
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install torch --index-url https://download.pytorch.org/whl/cpu
# point DATA_DIR in config.py at the unzipped Kaggle folder, then:
.\run_all.ps1
```

The second is a single self-contained notebook, HAR_pipeline_inline.ipynb, which has every line of code inlined (no imports from my own files). I made this specifically so it can be graded by just opening it and running the cells top to bottom. The very first cell installs the packages; after that, set DATA_DIR and run everything. The last cell writes outputs/submissions/submission_final.csv, which is the exact file I uploaded to Kaggle.

On the consistency point in the assignment: everything is seeded (SEED = 42 for numpy, random, scikit-learn, LightGBM, XGBoost, CatBoost and PyTorch), and every cross-validation split is a GroupKFold on user_id so a user never appears in both the training and validation side of a fold. The gradient-boosted tree models are fully deterministic; the neural models (CNN-LSTM, MLP) and the random kernels of ROCKET have minor run-to-run variation on CPU that is inherent to PyTorch and is not perfectly bit-reproducible. Re-running the pipeline therefore reproduces the same method and a public score within a few thousandths of the reported value; the file submission_final.csv in the repository is the exact submission that achieved 0.8267.

1. Preliminary analysis

Before writing any models I spent time just looking at the data, and a few things shaped every decision afterwards.

Each file is a five-minute recording that has already been summarised into 300 rows, one per second, with six columns. Three of them (`mean_x`, `mean_y`, `mean_z`) are the average acceleration on each axis during that second, and three (`std_x`, `std_y`, `std_z`) are the standard deviation within the second, which is really a measure of how much the wrist was moving at sub-second scale. Every file has exactly one activity label from 0 to 5.

A few physical details from the assignment are worth keeping in mind. The sensor is a 3-axis accelerometer worn on the wrist, the measurements are in units of g ($1g$ is about 9.81 m/s^2), and importantly the gravity component has not been removed from the signal. That last point matters: the mean channels therefore carry both the actual motion and a roughly-constant gravity vector that depends on how the wrist happens to be oriented. Since each user wears the device a bit differently, anything I built directly on the raw orientation of the mean channels was likely to be person-specific rather than activity-specific — which, as it turned out later, is exactly why an orientation-normalisation experiment hurt rather than helped (Section 4). The std channels, by contrast, only reflect how much the wrist moved, not which way it was pointing, so they are far more comparable across users.

The first useful fact is that every file is exactly 300 rows with no missing values, so I never had to deal with ragged sequences or imputation. The second fact is the one that mattered most: the 60 users in the training set and the 40 users in the test set do not overlap at all. That immediately told me a normal random train/validation split would be misleading, because the model could memorise a user and look great in validation while failing on genuinely new people. So I used GroupKFold by user everywhere, which forces validation onto users the model has never seen and mirrors the real train/test gap. This single choice is probably the most important one in the whole project.

The third fact is that the classes are very unevenly distributed. Classes 0 and 1 are about 42% of the files each, while class 2 is only ~3%, class 4 ~1%, and class 5 ~5%. Since the competition is scored on macro-F1, which averages the per-class F1 equally, those rare classes count just as much as the big ones. In practice that meant the rare classes, especially class 2, were where almost all of my score was being lost.

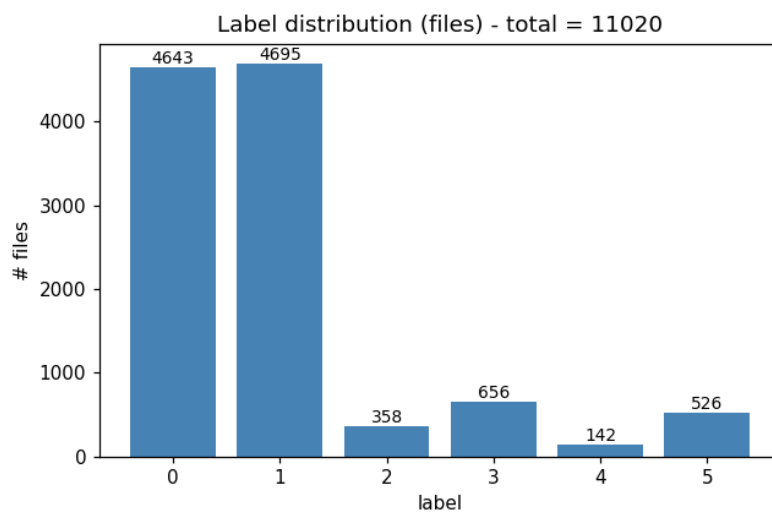


Figure 1. How many files belong to each activity class in the training set.

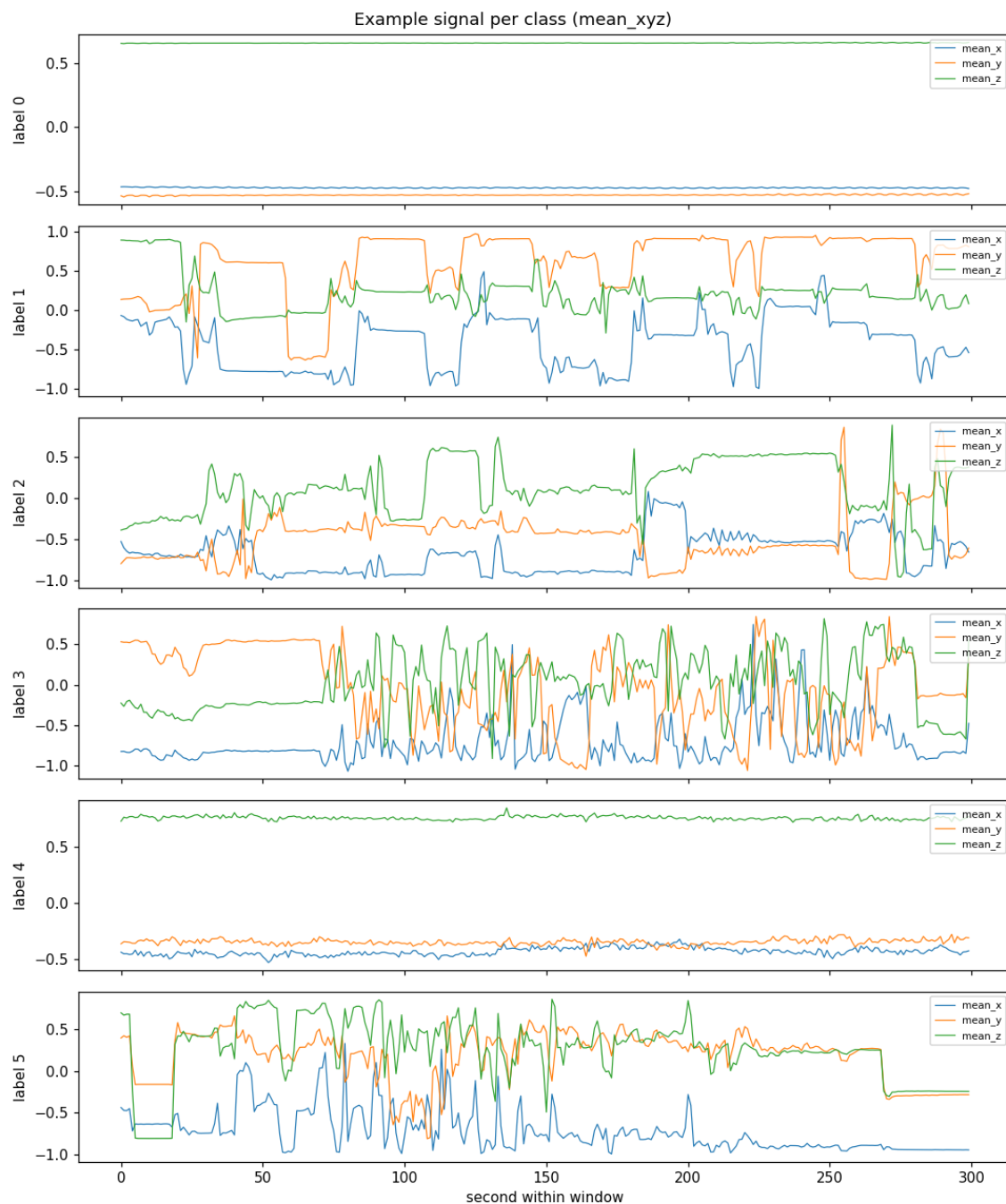


Figure 2. One example mean_x/y/z trace per class. The active classes have obvious oscillation; the static ones are almost flat.

To get a baseline number to improve on, I aggregated each file down to just 12 features (the mean and standard deviation over time of the six columns) and trained a logistic regression and a random forest with the same GroupKFold(user) setup. The better of the two reached a macro-F1 of 0.6281. That became my reference point. Looking at where it failed made the plan

obvious: the easy classes were already fine, and the only way to beat Baseline-3 (0.7088) was to capture WHEN things happen during the five minutes, not just the averages.

2. Preprocessing and feature engineering

Because the data is already aggregated to one-second rows, there was not much classical cleaning to do. I cache the raw data as parquet for speed, standardise the inputs per fold for the neural models (fitting the scaler only on the training side so nothing leaks), and use class-balanced sample weights so the rare classes are not ignored. The real work was building good per-file features.

I ended up with 334 features per file, organised into 17 named groups so I could switch each group on and off and measure it. Rather than list all 334, the groups are: basic statistics (mean/std/min/max/median), percentiles, skew and kurtosis, FFT band energy with spectral entropy and dominant frequency, mean-crossing counts, signal magnitude and signal-magnitude-area, cross-axis correlations and ratios, jerk (first differences over time), summaries of the std columns, autocorrelation at several lags, per-third temporal segment statistics, peak counts and spacing, energy and histogram entropy, distribution tails, and a small 'covmix' group of coefficient-of-variation and mean/std coupling terms.

The most valuable single change I made to the features came late and is worth calling out. Originally I only ran the rhythm-style extractors (autocorrelation, peak counts, tails) on the three mean channels. But the std channels are exactly the within-second motion intensity, and the rhythm of that intensity is basically step cadence. Running the same extractors over the std channels too is what pushed me from the low-0.81 range to 0.8267 on the leaderboard, and the interesting part is that it barely changed my cross-validation score. I think that is because step cadence is a property of the activity rather than of the person, so it transfers to new users far better than it shows up when validating on the same users.

The table below shows how the score grew as each model class was added, all measured with the same GroupKFold(user) cross-validation so they are comparable.

Stage	CV macro-F1	vs naive
Naive baseline (12 features, LogReg/RF)	0.6281	+0.0000
LightGBM (full feature set)	0.7154	+0.0873
XGBoost (full feature set)	0.7261	+0.0980
CatBoost (full feature set)	0.7269	+0.0988
MLP (full feature set)	0.7130	+0.0849
CNN-LSTM (raw 6x300 signal)	0.6337	+0.0056
ROCKET (random kernels, raw signal)	0.6669	+0.0388
Final 6-model ensemble	0.7586	+0.1305

In words: the feature engineering alone takes the gradient-boosted trees from 0.6281 up to around 0.7269, which is most of the total gain. Each extra model family then adds a little on top, and the final ensemble reaches 0.7586 in cross-validation.

Which feature groups actually mattered

To check that the groups were pulling their weight rather than just adding noise, I ran a leave-one-group-out study with LightGBM: drop one group, re-run the full cross-validation, and see how much the score falls. A positive number means the score got worse without that group, i.e. it carries unique signal.

Group	# feats	F1 using only this group	Drop when removed
energy	21	0.5988	+0.0036
autocorr	35	0.3986	+0.0017
covmix	10	0.6092	+0.0012
fft	42	0.3736	+0.0002
corr	3	0.2285	-0.0009
basic	30	0.6872	-0.0010
seg	54	0.6789	-0.0024
jerk	12	0.5020	-0.0025
mag	7	0.4245	-0.0027
crossings	6	0.4234	-0.0031
pct	30	0.6851	-0.0032
tails	28	0.6625	-0.0040
std_sum	9	0.6010	-0.0043
moments	12	0.4619	-0.0064
sma	4	0.3240	-0.0069
peaks	21	0.6102	-0.0085
ratio	10	0.3585	-0.0089

Two things stand out. First, no single group is decisive — the model leans on many correlated views of the same signal, so removing any one only costs a little. Second, the 'only this group' column is the more honest measure of raw strength, and there the basic statistics, percentiles, temporal segments and tails are clearly the strongest on their own.

3. Aligning the labels with the time series

A file is an ordered sequence of 300 seconds, so a good model has to use when things happen, not just the averaged statistics. I attacked this from three different directions on purpose, because the ensemble only benefits if the models make different kinds of mistakes.

Hand-crafted temporal features (for the trees and the MLP)

The most important one is autocorrelation at lags of 1, 2, 5, 10 and 20 seconds. A walking signal repeats at its stride interval, so its autocorrelation is high at that lag, while a static activity has none — this is a direct, scale-free measure of cadence, and as I mentioned it works especially well on the std channels. On top of that I add FFT band energy and dominant frequency (the frequency-domain view of the same rhythm), jerk and mean-crossing rates (how abrupt and how oscillatory the motion is), and per-third segment statistics that capture whether the activity changes over the five minutes.

A CNN-LSTM on the raw signal

I also trained a sequence model directly on the raw 6x300 input so it could learn temporal patterns I might not have hand-coded. The architecture is three 1D convolution blocks (which pick up local motion shapes and shorten the sequence), then a two-layer bidirectional LSTM over that, and finally average-and-max pooling over time before the classifier. The pooling makes it robust to where in the five minutes the activity is clearest. I train it with class weights, label smoothing, and light augmentation (small jitter, circular time-shifts and magnitude scaling) applied only to the training data, plus test-time augmentation when predicting. On its own it only scores about 0.634, but its mistakes are quite different from the trees, which is exactly why it helps the ensemble.

ROCKET (random convolutional kernels)

The third angle is ROCKET. It slides several thousand random convolutional filters over the raw signal and, for each one, records how often the response is positive and how large it gets, then a simple linear classifier sits on top. It is a well-known, fast time-series method, and I added it specifically because my three tree models all read the same hand-crafted features and were starting to make the same mistakes. ROCKET reads the raw signal in a completely different (random, linear) way, scores about 0.667 alone, and its errors are decorrelated from everything else, so it nudged the cross-validated ensemble up by roughly half a point.

4. Ablation study

The feature-group ablation is in Section 2. Here I cover the rest of the design choices, including — honestly — the things I tried that did not work, because they explain why the final system looks the way it does.

Comparing the model families

Model	CV macro-F1
Naive baseline (12 features, LogReg/RF)	0.6281
LightGBM (full feature set)	0.7154
XGBoost (full feature set)	0.7261
CatBoost (full feature set)	0.7269
MLP (full feature set)	0.7130
CNN-LSTM (raw 6x300 signal)	0.6337
ROCKET (random kernels, raw signal)	0.6669
Final 6-model ensemble	0.7586

How the models are combined

I compared two ways of combining the six models. One was a logistic-regression stacker trained on all of their out-of-fold probabilities, which reached about 0.7271. The other, and the one I kept, is a hill-climbing weighted blend: start from nothing and repeatedly add whichever model most improves the cross-validated macro-F1, so each model's number of picks becomes its weight. That gave 0.7530, a bit higher than the stacker, so I went with it. I use hill-climbing rather than a full weight grid simply because a grid over six models is far too large, and a nice

side effect is that it automatically down-weights the redundant tree models and leans on the diverse ones (the CNN, ROCKET and the MLP).

After blending I do one more step: a per-class additive offset, tuned greedily on the cross-validation folds, that shifts the decision boundary for the hard classes. This turned out to be one of the two biggest contributors to my final score — removing it dropped my leaderboard result from 0.8267 to 0.8110, so it is worth about +0.016 on its own. I tune it on the training folds rather than the full out-of-fold set so it generalises instead of memorising noise.

Where the score is lost: class 2

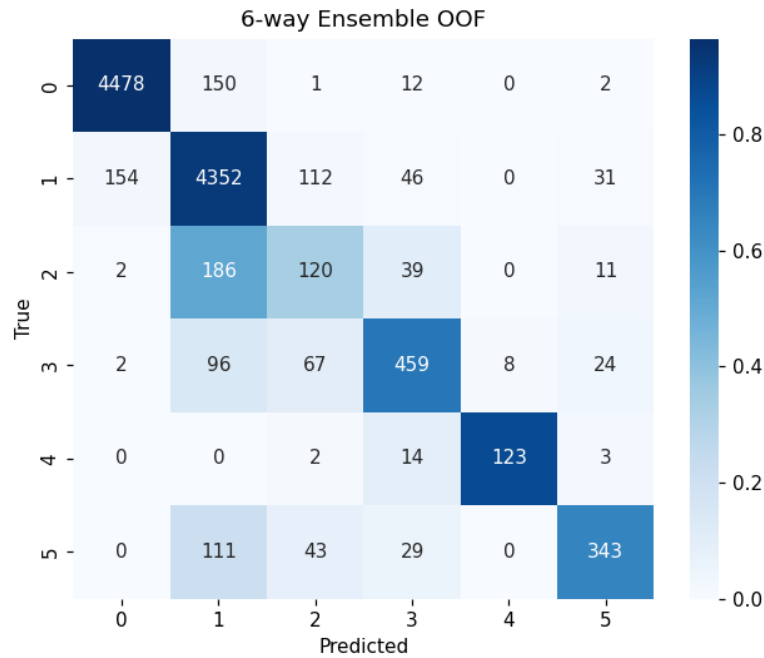


Figure 3. Confusion matrix of the final ensemble (out-of-fold).

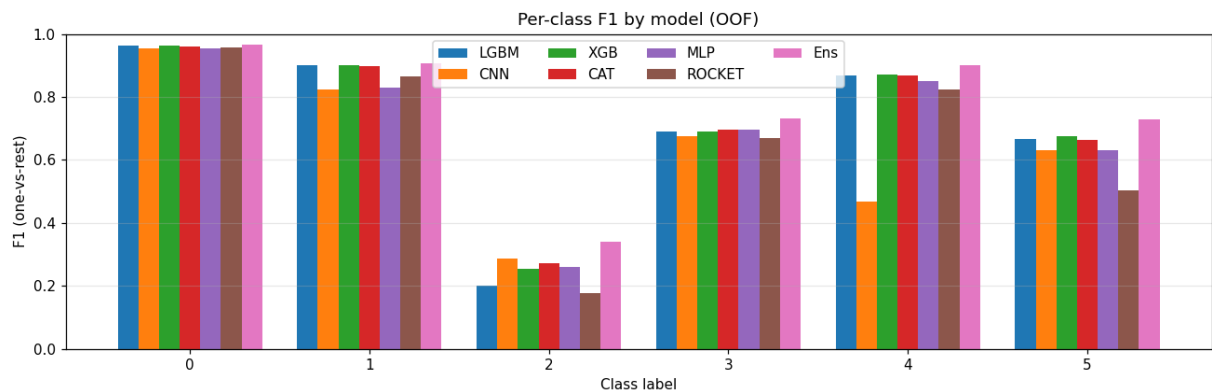


Figure 4. Per-class F1 for every model. Class 2 is the obvious weak spot across the board.

Per-class F1 of the final ensemble is 0.97 for class 0, 0.90 for class 1, 0.32 for class 2, 0.73 for class 3, 0.91 for class 4, 0.73 for class 5.

Almost all of the remaining error is class 2 being confused with class 1. I checked whether this was fixable: a classifier trained only to separate class 2 from class 1 still only reaches about 0.78 AUC, so the two activities genuinely look alike from the wrist, and in the full six-class problem the tiny class 2 (3% of the data) gets swamped by the huge class 1 (42%). Every other class is already strong, so this one confusion is the real ceiling for this dataset.

Things I tried that did not help

I think the negative results are as informative as the positive ones, so here is an honest summary of ideas I implemented, tested, and dropped:

What I tried	What happened	Why I think it failed
A dedicated class-2 vs class-1 'specialist' blended in	no change	same features, no new information to add
Gravity-aligned (orientation-normalised) features	0.8267 -> 0.8116	depends on device orientation, which differs per user
Multi-scale wavelet energy features	0.8267 -> 0.8024	redundant with what the trees already extract
Feeding ROCKET features into LightGBM (hybrid)	slightly worse	thousands of random columns dilute the tree splits
Matching the predicted class balance to the training balance	clearly worse	the test set genuinely is not distributed like train
Pseudo-labelling confident test predictions	-0.011 in validation	confident predictions are mostly the easy classes
A stronger / enriched CNN as a 7th model	0.8267 -> 0.8187	too correlated with the CNN already in the blend

The pattern I took away from all of this: once the tree models were saturated, no amount of recombination or extra correlated models helped. The only changes that ever moved the leaderboard were genuinely new, person-independent signal (the std-channel cadence features) and the per-class tuning. I also learned not to trust the cross-validation score blindly — a couple of ideas improved it slightly and then lost on the real test set, so I treated the leaderboard as the final judge.

5. Version history

This is the full record of every version I tried, in order. The first seven are the ones I kept, each building on the last up to the final 0.8267 submission. Everything after that is an idea I tried on top of the final model and reverted because it did not help on the leaderboard. Scores are the Kaggle public macro-F1 unless noted; a dash means I did not submit that version (either it was an intermediate build step or it lost in cross-validation before I spent a slot).

Version	What I did	Kept / failed	Kaggle public F1
v1	Naive baseline: 12 features (time mean+std), LogReg / Random Forest	kept (baseline)	CV 0.628
v2	Full 334-feature set + LightGBM	kept	—

v3	Added CNN-LSTM on the raw signal + weighted blend	kept	0.7739
v4	Added XGBoost as a 3rd model; expanded the feature set	kept	0.7919
v5	Added CatBoost + MLP, multi-seed bagging (5-model ensemble)	kept	0.8103
v6	Added ROCKET; switched to hill-climb blend (6-model ensemble)	kept	0.8115
v7	Cadence features on the std channels -> FINAL SUBMISSION	kept (final)	0.8267
e1	Gravity-aligned (orientation-normalised) features	failed, reverted	0.8116
e2	Multi-scale Haar wavelet energy features	failed, reverted	0.8024
e3	Enriched CNN added as a 7th model (also tried replacing the CNN)	failed, reverted	0.8187 / 0.8091
e4	Stronger ROCKET (6000 kernels) swapped in / added	failed, reverted	no improvement
e5	Pushed the class-2 offset higher (+0.05 / +0.10)	failed	0.8181 / 0.8143
e6	Matched predicted class balance to the training priors	failed	0.79 - 0.81
e7	Ensemble variants: equal-weight, single model, trees-only, no tuning	failed	0.78 - 0.81
e8	Pseudo-labelling confident test predictions	failed (CV -0.011)	not submitted
e9	Frequency-domain (power-spectrum) model	failed (CV 0.45)	not submitted

The version I am most pleased with is v7. It came from a small, well-reasoned feature idea — measuring cadence on the std channels — rather than from throwing more models at the problem, and it is the step that moved me from the middle of the leaderboard up near the top. Everything after v7 confirmed that the model had reached its ceiling: nine further ideas, all reverted, none beating 0.8267.

6. Summary

My final submission is a six-model ensemble — LightGBM [3], XGBoost [2], CatBoost [4] and an MLP on 334 hand-crafted features, plus a CNN-LSTM [8], [9] and a ROCKET [5] model on the raw signal — combined with a hill-climbing weighted blend [7] and a per-class offset correction. The implementation uses scikit-learn [6], and the dataset is the wrist-accelerometer ADL dataset of Bruno et al. [1]. The system scores 0.7586 in GroupKFold(user) cross-validation and 0.8267 on the public leaderboard, which comfortably clears the Baseline-3 target of 0.7088. Everything is seeded and reproducible from either run_all.ps1 or the inline notebook, and the file it produces (submission_final.csv) is the one I submitted.

References

- [1] B. Bruno, F. Mastrogiovanni, and A. Sgorbissa, “A public domain dataset for ADL recognition using wrist-placed accelerometers,” in Proc. 23rd IEEE Int. Symp. Robot and Human Interactive Communication (RO-MAN), 2014, pp. 738–743.
- [2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

- [3] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [4] L. Prokhorenkova et al., "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
- [5] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020.
- [6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [7] R. Caruana, A. Niculescu-Mizil, G. Crew, and A. Ksikes, "Ensemble selection from libraries of models," in *Proc. 21st Int. Conf. Machine Learning (ICML)*, 2004.
- [8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [9] F. J. Ordóñez and D. Roggen, "Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, p. 115, 2016.